AI Chatbot Implementation Blueprint

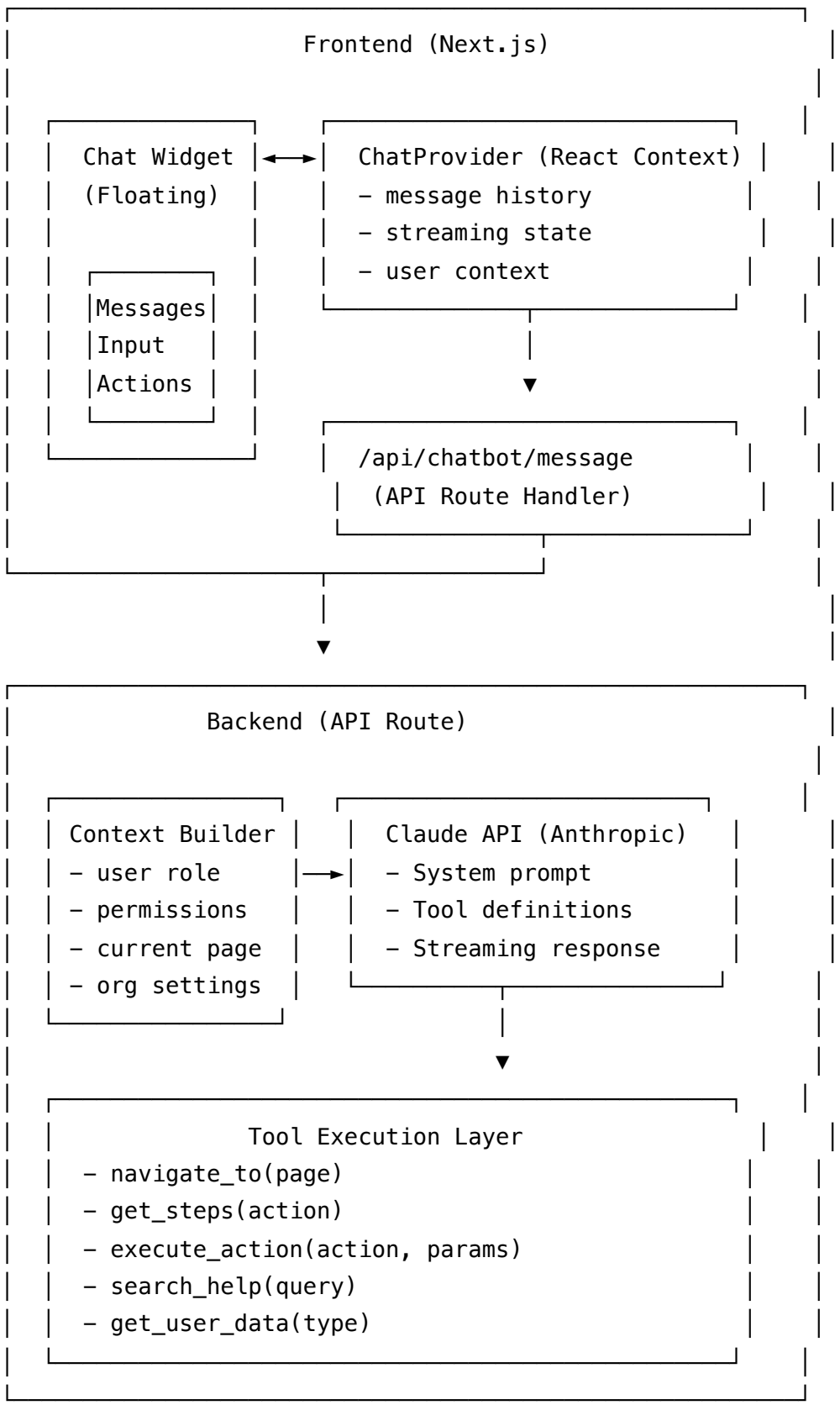
Tatwir Talent HRMS - Intelligent Assistant

1. Overview

An AI-powered chatbot embedded in the Tatwir Talent HRMS that guides users through every system operation. The chatbot:

- **Understands context**: Knows the user's role, permissions, and current page
- **Provides step-by-step guidance**: Breaks down any task into actionable steps
- **Executes actions on behalf**: When the user confirms, the bot can trigger API calls directly
- **Speaks Arabic & English**: Bilingual support for Saudi workforce

2. Architecture



3. Tech Stack for Chatbot

Component	Technology	Reason
AI Engine	Claude API (Anthropic)	Tool use, streaming, Arabic support
Frontend Widget	React + Tailwind + shadcn/ui	Matches existing UI system
State Management	React Context + useReducer	Lightweight, no extra deps
Streaming	Server-Sent Events (SSE)	Real-time token streaming
Message Storage	Supabase (PostgreSQL)	Conversation history persistence
Rate Limiting	Existing rate-limit system	Prevent API abuse
Auth	Existing tenant context	Same auth flow as all other APIs

4. Database Schema

```sql
-- Migration: schema_v137_chatbot.sql

-- Conversation sessions
CREATE TABLE chatbot_conversations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  organization_id UUID NOT NULL REFERENCES organizations(id),
  user_id UUID NOT NULL REFERENCES auth.users(id),
  title TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  is_archived BOOLEAN NOT NULL DEFAULT false
);

-- Individual messages
CREATE TABLE chatbot_messages (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  conversation_id UUID NOT NULL REFERENCES chatbot_conversations(id) ON DELETE CASCADE,
  role TEXT NOT NULL CHECK (role IN ('user', 'assistant', 'system')),
  content TEXT NOT NULL,
  metadata JSONB DEFAULT '{}',
  -- metadata stores: tool_calls, action_results, step_progress, etc.
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Action execution audit log
CREATE TABLE chatbot_actions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  conversation_id UUID NOT NULL REFERENCES chatbot_conversations(id),
  message_id UUID NOT NULL REFERENCES chatbot_messages(id),
  action_type TEXT NOT NULL,
  action_params JSONB NOT NULL DEFAULT '{}',
  status TEXT NOT NULL CHECK (status IN ('pending', 'confirmed', 'executed', 'failed',
  result JSONB,
  executed_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Indexes
CREATE INDEX idx_chatbot_conversations_user ON chatbot_conversations(user_id, organizat
CREATE INDEX idx_chatbot_messages_conversation ON chatbot_messages(conversation_id, cre
```

```sql
CREATE INDEX idx_chatbot_actions_conversation ON chatbot_actions(conversation_id);

-- RLS Policies
ALTER TABLE chatbot_conversations ENABLE ROW LEVEL SECURITY;
ALTER TABLE chatbot_messages ENABLE ROW LEVEL SECURITY;
ALTER TABLE chatbot_actions ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Users see own conversations"
  ON chatbot_conversations FOR ALL
  USING (user_id = auth.uid());

CREATE POLICY "Users see own messages"
  ON chatbot_messages FOR ALL
  USING (conversation_id IN (
    SELECT id FROM chatbot_conversations WHERE user_id = auth.uid()
  ));

CREATE POLICY "Users see own actions"
  ON chatbot_actions FOR ALL
  USING (conversation_id IN (
    SELECT id FROM chatbot_conversations WHERE user_id = auth.uid()
  ));
```

5. File Structure

```
src/
├── app/
│   └── api/
│       └── chatbot/
│           ├── message/
│           │   └── route.ts          # Main chat endpoint (streaming)
│           ├── conversations/
│           │   └── route.ts          # List/create conversations
│           ├── conversations/
│           │   └── [id]/
│           │       └── route.ts      # Get/archive conversation
│           └── actions/
│               └── [id]/
│                   └── confirm/
│                       └── route.ts  # Confirm/execute pending action
├── components/
│   └── chatbot/
│       ├── ChatWidget.tsx            # Floating chat bubble + panel
│       ├── ChatPanel.tsx             # Main chat interface
│       ├── ChatMessage.tsx           # Single message renderer
│       ├── ChatInput.tsx             # Message input with suggestions
│       ├── ActionCard.tsx            # Rendered action confirmation card
│       ├── StepGuide.tsx             # Step-by-step guide renderer
│       └── ChatProvider.tsx          # Context provider
├── lib/
│   ├── chatbot/
│   │   ├── system-prompt.ts          # System prompt builder
│   │   ├── tools.ts                  # Claude tool definitions
│   │   ├── tool-handlers.ts         # Tool execution logic
│   │   ├── context-builder.ts       # User/page context assembly
│   │   ├── action-registry.ts       # All executable actions registry
│   │   └── knowledge-base.ts        # System knowledge for guidance
│   └── services/
│       └── chatbot.ts               # Client-side service
└── hooks/
    └── use-chatbot.ts               # Chat hook for components
```

6. System Prompt Design

```
// src/lib/chatbot/system-prompt.ts

export function buildSystemPrompt(context: ChatContext): string {
  return `You are the Tatwir Talent HR Assistant, an AI embedded in the Tatwir Talent H

## Your Role
You help users navigate and use the HR system effectively. You can:
1. Explain how to perform any action in the system
2. Provide step-by-step walkthroughs
3. Execute actions on the user's behalf (with their confirmation)
4. Answer questions about HR policies, system features, and data

## Current User Context
- Name: ${context.userName}
- Role: ${context.userRole}
- Organization: ${context.orgName}
- Current Page: ${context.currentPage}
- Language Preference: ${context.language}
- Permissions: ${context.permissions.join(', ')}

## System Modules Available to This User
${context.availableModules.map(m => `- ${m.name}: ${m.description}`).join('\n')}

## Behavioral Rules
1. Always respond in the user's language (Arabic or English). If they write in Arabic,
2. When explaining steps, number them clearly and include the exact UI element names.
3. Before executing any action, always show what you're about to do and ask for confirm
4. If the user doesn't have permission for something, explain what role/permission is n
5. Be concise but helpful. Don't over-explain simple things.
6. When providing steps, include navigation: "Go to [Module] > [Page] > [Action]"
7. If you're unsure about something, say so rather than guessing.

## Action Execution
When the user wants you to DO something (not just explain):
1. Use the appropriate tool to prepare the action
2. Present the action details in a confirmation card
3. Only execute after the user confirms
4. Report the result clearly

## Knowledge Base
```

```
${context.knowledgeBase}
`;
}
```

7. Claude Tool Definitions

```
// src/lib/chatbot/tools.ts

import Anthropic from '@anthropic-ai/sdk';

export const chatbotTools: Anthropic.Tool[] = [
  {
    name: 'get_step_by_step_guide',
    description: 'Get a detailed step-by-step guide for performing an action in the sys
    input_schema: {
      type: 'object',
      properties: {
        action: {
          type: 'string',
          description: 'The action the user wants to perform (e.g., "add new employee",
        },
        user_role: {
          type: 'string',
          description: 'The current user role for permission-aware guidance'
        }
      },
      required: ['action']
    }
  },
  {
    name: 'navigate_user',
    description: 'Guide or navigate the user to a specific page in the system.',
    input_schema: {
      type: 'object',
      properties: {
        destination: {
          type: 'string',
          description: 'The page/module to navigate to'
        },
        reason: {
          type: 'string',
          description: 'Why the user should go there'
        }
      },
      required: ['destination']
    }
```

```
},
{
  name: 'execute_system_action',
  description: 'Execute an action in the system on behalf of the user. ALWAYS ask for
  input_schema: {
    type: 'object',
    properties: {
      action_type: {
        type: 'string',
        enum: [
          'create_employee',
          'update_employee',
          'submit_leave_request',
          'approve_leave_request',
          'reject_leave_request',
          'create_task',
          'update_task',
          'check_in_attendance',
          'check_out_attendance',
          'create_department',
          'create_location',
          'create_shift',
          'assign_shift',
          'run_payroll',
          'create_job_posting',
          'move_application_stage',
          'create_asset',
          'submit_advance_request'
        ],
        description: 'The type of action to execute'
      },
      params: {
        type: 'object',
        description: 'Parameters for the action (varies by action_type)'
      },
      confirmation_message: {
        type: 'string',
        description: 'A human-readable summary of what will be done, shown to user fo
      }
    },
    required: ['action_type', 'params', 'confirmation_message']
  }
},
```

```
{
  name: 'query_system_data',
  description: 'Query data from the system to answer user questions (e.g., "how many
  input_schema: {
    type: 'object',
    properties: {
      query_type: {
        type: 'string',
        enum: [
          'employee_count',
          'employee_list',
          'employee_details',
          'leave_balance',
          'leave_requests',
          'attendance_today',
          'attendance_summary',
          'pending_tasks',
          'payroll_status',
          'department_list',
          'location_list',
          'shift_schedule',
          'open_job_postings',
          'pending_approvals',
          'notifications_unread'
        ],
        description: 'Type of data to query'
      },
      filters: {
        type: 'object',
        description: 'Optional filters (e.g., department_id, date_range, employee_id)
      }
    },
    required: ['query_type']
  }
},
{
  name: 'search_help',
  description: 'Search the knowledge base for help articles and system documentation.
  input_schema: {
    type: 'object',
    properties: {
      query: {
        type: 'string',
```

```
          description: 'Search query for help content'
        }
      },
      required: ['query']
    }
  }
];
```

8. API Route Implementation

```
// src/app/api/chatbot/message/route.ts

import { NextRequest } from 'next/server';
import Anthropic from '@anthropic-ai/sdk';
import { requireTenantContext } from '@/lib/server/tenant-context';
import { buildSystemPrompt } from '@/lib/chatbot/system-prompt';
import { chatbotTools } from '@/lib/chatbot/tools';
import { handleToolCall } from '@/lib/chatbot/tool-handlers';
import { buildChatContext } from '@/lib/chatbot/context-builder';

const anthropic = new Anthropic({
  apiKey: process.env.ANTHROPIC_API_KEY!,
});

export async function POST(req: NextRequest) {
  const tenant = await requireTenantContext(req);
  if ('error' in tenant) {
    return Response.json({ error: tenant.error }, { status: tenant.status });
  }

  const { messages, conversationId, currentPage } = await req.json();

  // Build context
  const context = await buildChatContext({
    userId: tenant.userId,
    orgId: tenant.orgId,
    role: tenant.role,
    currentPage,
    adminClient: tenant.admin,
  });

  const systemPrompt = buildSystemPrompt(context);

  // Save user message to DB
  await tenant.admin
    .from('chatbot_messages')
    .insert({
      conversation_id: conversationId,
      role: 'user',
      content: messages[messages.length - 1].content,
```

```
  });

// Create streaming response
const encoder = new TextEncoder();
const stream = new ReadableStream({
  async start(controller) {
    try {
      let currentMessages = messages;
      let continueLoop = true;

      while (continueLoop) {
        const response = await anthropic.messages.create({
          model: 'claude-sonnet-5',
          max_tokens: 4096,
          system: systemPrompt,
          tools: chatbotTools,
          messages: currentMessages,
          stream: true,
        });

        let fullResponse = '';
        let toolUseBlocks: any[] = [];
        let currentToolUse: any = null;

        for await (const event of response) {
          if (event.type === 'content_block_start') {
            if (event.content_block.type === 'text') {
              // Stream text
            } else if (event.content_block.type === 'tool_use') {
              currentToolUse = {
                id: event.content_block.id,
                name: event.content_block.name,
                input: '',
              };
            }
          } else if (event.type === 'content_block_delta') {
            if (event.delta.type === 'text_delta') {
              fullResponse += event.delta.text;
              controller.enqueue(
                encoder.encode(`data: ${JSON.stringify({ type: 'text', content: event
              );
            } else if (event.delta.type === 'input_json_delta') {
              if (currentToolUse) {
```

```
        currentToolUse.input += event.delta.partial_json;
      }
    }
  } else if (event.type === 'content_block_stop') {
    if (currentToolUse) {
      currentToolUse.input = JSON.parse(currentToolUse.input);
      toolUseBlocks.push(currentToolUse);
      currentToolUse = null;
    }
  } else if (event.type === 'message_delta') {
    if (event.delta.stop_reason === 'tool_use') {
      // Handle tool calls
      const toolResults = [];
      for (const toolUse of toolUseBlocks) {
        // Notify frontend about tool execution
        controller.enqueue(
          encoder.encode(`data: ${JSON.stringify({
            type: 'tool_call',
            tool: toolUse.name,
            input: toolUse.input,
          })}\n\n`)
        );

        const result = await handleToolCall(
          toolUse.name,
          toolUse.input,
          tenant
        );

        toolResults.push({
          type: 'tool_result',
          tool_use_id: toolUse.id,
          content: JSON.stringify(result),
        });

        // Notify frontend about tool result
        controller.enqueue(
          encoder.encode(`data: ${JSON.stringify({
            type: 'tool_result',
            tool: toolUse.name,
            result,
          })}\n\n`)
        );
```

```
            }

            // Continue conversation with tool results
            currentMessages = [
              ...currentMessages,
              { role: 'assistant', content: [...(fullResponse ? [{ type: 'text', te
              ...toolResults.map(r => ({ role: 'user', content: [r] })),
            ];

            fullResponse = '';
            toolUseBlocks = [];
          } else {
            continueLoop = false;
          }
        }
      }

      // Save assistant response
      await tenant.admin
        .from('chatbot_messages')
        .insert({
          conversation_id: conversationId,
          role: 'assistant',
          content: fullResponse,
        });

      controller.enqueue(encoder.encode(`data: ${JSON.stringify({ type: 'done' })}\n\
      controller.close();
    } catch (error: any) {
      controller.enqueue(
        encoder.encode(`data: ${JSON.stringify({ type: 'error', message: error.messag
      );
      controller.close();
    }
  },
});

return new Response(stream, {
  headers: {
    'Content-Type': 'text/event-stream',
    'Cache-Control': 'no-cache',
    Connection: 'keep-alive',
```

```
    },
  });
}
```

9. Knowledge Base (Action Registry)

```
// src/lib/chatbot/knowledge-base.ts

export interface ActionGuide {
  id: string;
  title: string;
  titleAr: string;
  module: string;
  requiredRoles: string[];
  steps: Step[];
  stepsAr: Step[];
  route: string;
}

export interface Step {
  order: number;
  instruction: string;
  uiElement?: string;
  tip?: string;
}

export const actionGuides: ActionGuide[] = [
  // === EMPLOYEE MANAGEMENT ===
  {
    id: 'add_employee',
    title: 'Add a New Employee',
    titleAr: 'إضافة موظف جديد',
    module: 'HR',
    requiredRoles: ['Superadmin', 'Admin', 'HR_Admin'],
    route: '/employees',
    steps: [
      { order: 1, instruction: 'Navigate to "Employees" from the sidebar menu', uiEleme
      { order: 2, instruction: 'Click the "Add Employee" button in the top right', uiEl
      { order: 3, instruction: 'Fill in the required fields: Full Name, Employee ID, De
      { order: 4, instruction: 'Optionally fill in: Phone, Email, Nationality, ID Numbe
      { order: 5, instruction: 'Set the salary details: Basic Salary, Housing Allowance
      { order: 6, instruction: 'Click "Save" to create the employee record', uiElement:
      { order: 7, instruction: 'The system will create a login account if email is prov
    ],
    stepsAr: [
      { order: 1, instruction: 'انتقل إلى "الموظفين" من القائمة الجانبية', uiElement: '
```

```
    { order: 2, instruction: 'اضغط على زر "إضافة موظف" في أعلى اليمين', uiElement: 'ف
    { order: 3, instruction: 'الاسم الكامل، رقم الموظف، القسم، المنصب، تاريخ التعيين :
    { order: 4, instruction: 'تف، البريد الإلكتروني، الجنسية، رقم الهوية، تفاصيل البنك
    { order: 5, instruction: 'حدد تفاصيل الراتب: الراتب الأساسي، بدل السكن، بدل النقل
    { order: 6, instruction: 'اضغط "حفظ" لإنشاء سجل الموظف', uiElement: 'زر: حفظ' },
    { order: 7, instruction: 'م النظام بإنشاء حساب دخول إذا تم توفير البريد الإلكتروني
  ],
},
{
  id: 'submit_leave_request',
  title: 'Submit a Leave Request',
  titleAr: 'تقديم طلب إجازة',
  module: 'Leave',
  requiredRoles: ['Employee', 'Admin', 'HR_Admin', 'Sales', 'Sales_Manager', 'Finance
  route: '/leaves',
  steps: [
    { order: 1, instruction: 'Navigate to "Leaves" from the sidebar menu', uiElement:
    { order: 2, instruction: 'Click "Request Leave" button', uiElement: 'Button: Requ
    { order: 3, instruction: 'Select the leave type (Annual, Sick, Emergency, etc.)'
    { order: 4, instruction: 'Choose the start date and end date' },
    { order: 5, instruction: 'Add a reason/note for the leave' },
    { order: 6, instruction: 'Click "Submit" to send for approval', uiElement: 'Butto
    { order: 7, instruction: 'Your manager or HR will review and approve/reject the r
  ],
  stepsAr: [
    { order: 1, instruction: 'انتقل إلى "الإجازات" من القائمة الجانبية', uiElement: 'ت
    { order: 2, instruction: 'اضغط على زر "طلب إجازة"', uiElement: 'زر: طلب إجازة' },
    { order: 3, instruction: 'اختر نوع الإجازة (سنوية، مرضية، طارئة، إلخ)' },
    { order: 4, instruction: 'حدد تاريخ البداية وتاريخ النهاية' },
    { order: 5, instruction: 'أضف سبب/ملاحظة للإجازة' },
    { order: 6, instruction: 'اضغط "إرسال" لتقديم الطلب للموافقة', uiElement: 'إرسال
    { order: 7, instruction: 'مديرك أو الموارد البشرية بمراجعة الطلب والموافقة/الرفض
  ],
},
{
  id: 'approve_leave',
  title: 'Approve or Reject a Leave Request',
  titleAr: 'الموافقة على أو رفض طلب إجازة',
  module: 'Leave',
  requiredRoles: ['Admin', 'HR_Admin'],
  route: '/leaves',
  steps: [
    { order: 1, instruction: 'Navigate to "Leaves" from the sidebar menu' },
```

```
    { order: 2, instruction: 'Find the pending leave request in the list (status: Pen
    { order: 3, instruction: 'Click on the request to view details' },
    { order: 4, instruction: 'Review the dates, type, reason, and remaining balance'
    { order: 5, instruction: 'Click "Approve" to approve or "Reject" with a mandatory
  ],
  stepsAr: [
    { order: 1, instruction: 'انتقل إلى "الإجازات" من القائمة الجانبية' },
    { order: 2, instruction: 'ابحث عن طلب الإجازة المعلق في القائمة (الحالة: معلق)' },
    { order: 3, instruction: 'اضغط على الطلب لعرض التفاصيل' },
    { order: 4, instruction: 'راجع التواريخ، النوع، السبب، والرصيد المتبقي' },
    { order: 5, instruction: 'اضغط "موافقة" للموافقة أو "رفض" مع ملاحظة إلزامية', tip:
  ],
},
{
  id: 'check_in_attendance',
  title: 'Check In for Attendance',
  titleAr: 'تسجيل حضور',
  module: 'Attendance',
  requiredRoles: ['Employee', 'Sales', 'Field_Agent', 'Promoter'],
  route: '/attendance',
  steps: [
    { order: 1, instruction: 'Navigate to "Attendance" from the sidebar menu' },
    { order: 2, instruction: 'Click the "Check In" button' },
    { order: 3, instruction: 'Allow location access when prompted by the browser' },
    { order: 4, instruction: 'Take a selfie when the camera opens (if required)' },
    { order: 5, instruction: 'The system will validate your GPS location against your
    { order: 6, instruction: 'Once validated, your check-in time is recorded', tip: '
  ],
  stepsAr: [
    { order: 1, instruction: 'انتقل إلى "الحضور" من القائمة الجانبية' },
    { order: 2, instruction: 'اضغط على زر "تسجيل حضور"' },
    { order: 3, instruction: 'اسمح بالوصول إلى الموقع عند طلب المتصفح' },
    { order: 4, instruction: 'التقط صورة سيلفي عند فتح الكاميرا (إذا مطلوب)' },
    { order: 5, instruction: 'سيتحقق النظام من موقع GPS الخاص بك مقابل موقع العمل المحدد
    { order: 6, instruction: 'بمجرد التحقق، يتم تسجيل وقت حضورك', tip: 'في لموقع عملك'
  ],
},
{
  id: 'run_payroll',
  title: 'Run Monthly Payroll',
  titleAr: 'تشغيل الرواتب الشهرية',
  module: 'Payroll',
  requiredRoles: ['Admin', 'HR_Admin', 'Finance_Manager'],
```

```
    route: '/payroll',
    steps: [
      { order: 1, instruction: 'Navigate to "Payroll" from the sidebar menu' },
      { order: 2, instruction: 'Select the target month and year' },
      { order: 3, instruction: 'Click "Run Payroll" to calculate all employee payslips' },
      { order: 4, instruction: 'Review the generated payslips for accuracy' },
      { order: 5, instruction: 'Make any manual adjustments if needed (bonus/deduction)' },
      { order: 6, instruction: 'Transition status from Draft → Approved → Paid' },
    ],
    stepsAr: [
      { order: 1, instruction: 'انتقل إلى "الرواتب" من القائمة الجانبية' },
      { order: 2, instruction: 'اختر الشهر والسنة المستهدفة' },
      { order: 3, instruction: 'اضغط "تشغيل الرواتب" لحساب كشوف رواتب جميع الموظفين' },
      { order: 4, instruction: 'راجع كشوف الرواتب المُنشأة للتأكد من دقتها' },
      { order: 5, instruction: 'قم بأي تعديلات يدوية إذا لزم الأمر (مكافأة/خصم)' },
      { order: 6, instruction: 'انقل الحالة من مسودة ← معتمد ← مدفوع' },
    ],
  },
  {
    id: 'create_task',
    title: 'Create and Assign a Task',
    titleAr: 'إنشاء وتعيين مهمة',
    module: 'Tasks',
    requiredRoles: ['Admin', 'HR_Admin', 'Sales_Manager', 'Onground_Agent_Manager'],
    route: '/tasks',
    steps: [
      { order: 1, instruction: 'Navigate to "Tasks" from the sidebar menu' },
      { order: 2, instruction: 'Click "Create Task" button' },
      { order: 3, instruction: 'Enter the task title and description' },
      { order: 4, instruction: 'Set the priority (Low, Medium, High, Critical)' },
      { order: 5, instruction: 'Choose the due date' },
      { order: 6, instruction: 'Assign to one or more employees' },
      { order: 7, instruction: 'Optionally attach files' },
      { order: 8, instruction: 'Click "Save" to create and notify the assigned employee' },
    ],
    stepsAr: [
      { order: 1, instruction: 'انتقل إلى "المهام" من القائمة الجانبية' },
      { order: 2, instruction: 'اضغط على زر "إنشاء مهمة"' },
      { order: 3, instruction: 'أدخل عنوان المهمة والوصف' },
      { order: 4, instruction: 'حدد الأولوية (منخفض، متوسط، عالي، حرج)' },
      { order: 5, instruction: 'اختر تاريخ الاستحقاق' },
      { order: 6, instruction: 'عيّن لموظف واحد أو أكثر' },
      { order: 7, instruction: 'اختياريًا أرفق ملفات' },
```

```
    { order: 8, instruction: 'اضغط "حفظ" لإنشاء وإشعار الموظفين المعينين' },
  ],
},
{
  id: 'create_job_posting',
  title: 'Create a Job Posting',
  titleAr: 'إنشاء إعلان وظيفي',
  module: 'Recruitment',
  requiredRoles: ['Admin', 'HR_Admin', 'HR_Recruitment'],
  route: '/recruitment',
  steps: [
    { order: 1, instruction: 'Navigate to "Recruitment" from the sidebar menu' },
    { order: 2, instruction: 'Click "Create Posting" button' },
    { order: 3, instruction: 'Enter the job title, department, and location' },
    { order: 4, instruction: 'Write the job description and requirements' },
    { order: 5, instruction: 'Configure the application form (custom fields if needed
    { order: 6, instruction: 'Set the posting status to "Open"' },
    { order: 7, instruction: 'Share the public application link with candidates' },
  ],
  stepsAr: [
    { order: 1, instruction: 'انتقل إلى "التوظيف" من القائمة الجانبية' },
    { order: 2, instruction: 'اضغط على زر "إنشاء إعلان"' },
    { order: 3, instruction: 'أدخل المسمى الوظيفي، القسم، والموقع' },
    { order: 4, instruction: 'اكتب وصف الوظيفة والمتطلبات' },
    { order: 5, instruction: 'قم بتكوين نموذج التقديم (حقول مخصصة إذا لزم)' },
    { order: 6, instruction: 'اضبط حالة الإعلان إلى "مفتوح"' },
    { order: 7, instruction: 'شارك رابط التقديم العام مع المرشحين' },
  ],
},
{
  id: 'log_daily_sales',
  title: 'Log Daily Sales',
  titleAr: 'تسجيل المبيعات اليومية',
  module: 'Sales',
  requiredRoles: ['Sales', 'Sales_Manager'],
  route: '/sales',
  steps: [
    { order: 1, instruction: 'Navigate to "Sales" from the sidebar menu' },
    { order: 2, instruction: 'Click "Add Sales Entry" or find today\'s entry' },
    { order: 3, instruction: 'Select the company and product' },
    { order: 4, instruction: 'Enter the quantity and any serial numbers' },
    { order: 5, instruction: 'Submit the entry' },
    { order: 6, instruction: 'To edit a submitted entry, submit an "Edit Request" for
```

```
    ],
    stepsAr: [
      { order: 1, instruction: 'انتقل إلى "المبيعات" من القائمة الجانبية' },
      { order: 2, instruction: 'اضغط "إضافة قيد مبيعات" أو ابحث عن قيد اليوم' },
      { order: 3, instruction: 'اختر الشركة والمنتج' },
      { order: 4, instruction: 'أدخل الكمية وأي أرقام تسلسلية' },
      { order: 5, instruction: 'أرسل القيد' },
      { order: 6, instruction: 'لتعديل قيد مرسل، قدم "طلب تعديل" لموافقة المدير' },
    ],
  },
  // ... More guides would be added for every system action
];
```

10. Frontend Chat Widget

```tsx
// src/components/chatbot/ChatWidget.tsx

'use client';

import { useState } from 'react';
import { MessageCircle, X, Minimize2 } from 'lucide-react';
import { ChatPanel } from './ChatPanel';
import { ChatProvider } from './ChatProvider';

export function ChatWidget() {
  const [isOpen, setIsOpen] = useState(false);
  const [isMinimized, setIsMinimized] = useState(false);

  return (
    <ChatProvider>
      {/* Floating Action Button */}
      {!isOpen && (
        <button
          onClick={() => setIsOpen(true)}
          className="fixed bottom-6 right-6 z-50 flex h-14 w-14 items-center justify-ce
          aria-label="Open chat assistant"
        >
          <MessageCircle className="h-6 w-6" />
        </button>
      )}

      {/* Chat Panel */}
      {isOpen && (
        <div
          className={`fixed bottom-6 right-6 z-50 flex flex-col rounded-2xl border bg-b
            isMinimized ? 'h-14 w-80' : 'h-[600px] w-[400px]'
          }`}
        >
          {/* Header */}
          <div className="flex items-center justify-between rounded-t-2xl border-b bg-p
            <div className="flex items-center gap-2">
              <div className="h-2 w-2 rounded-full bg-green-400 animate-pulse" />
              <span className="font-medium text-primary-foreground text-sm">
                Tatwir Assistant
              </span>
```

```
            </div>
            <div className="flex items-center gap-1">
              <button
                onClick={() => setIsMinimized(!isMinimized)}
                className="rounded p-1 text-primary-foreground/70 hover:text-primary-fo
              >
                <Minimize2 className="h-4 w-4" />
              </button>
              <button
                onClick={() => setIsOpen(false)}
                className="rounded p-1 text-primary-foreground/70 hover:text-primary-fo
              >
                <X className="h-4 w-4" />
              </button>
            </div>
          </div>

          {/* Chat Body */}
          {!isMinimized && <ChatPanel />}
        </div>
      )}
    </ChatProvider>
  );
}
```

11. Context Provider

```tsx
// src/components/chatbot/ChatProvider.tsx

'use client';

import { createContext, useContext, useReducer, useCallback, ReactNode } from 'react';
import { usePathname } from 'next/navigation';

interface Message {
  id: string;
  role: 'user' | 'assistant' | 'system';
  content: string;
  timestamp: Date;
  toolCalls?: ToolCall[];
  actionCard?: ActionCard;
  stepGuide?: StepGuide;
}

interface ToolCall {
  name: string;
  input: any;
  result?: any;
  status: 'pending' | 'completed' | 'error';
}

interface ActionCard {
  actionType: string;
  description: string;
  params: Record<string, any>;
  status: 'pending_confirmation' | 'confirmed' | 'executed' | 'cancelled';
}

interface StepGuide {
  title: string;
  steps: { order: number; instruction: string; completed: boolean }[];
  currentStep: number;
}

interface ChatState {
  messages: Message[];
  isLoading: boolean;
```

```
  conversationId: string | null;
  error: string | null;
}

type ChatAction =
  | { type: 'ADD_MESSAGE'; payload: Message }
  | { type: 'UPDATE_MESSAGE'; payload: { id: string; updates: Partial<Message> } }
  | { type: 'SET_LOADING'; payload: boolean }
  | { type: 'SET_CONVERSATION_ID'; payload: string }
  | { type: 'SET_ERROR'; payload: string | null }
  | { type: 'CLEAR_MESSAGES' };

const ChatContext = createContext<{
  state: ChatState;
  sendMessage: (content: string) => Promise<void>;
  confirmAction: (messageId: string) => Promise<void>;
  cancelAction: (messageId: string) => void;
  markStepComplete: (messageId: string, stepOrder: number) => void;
  clearChat: () => void;
} | null>(null);

function chatReducer(state: ChatState, action: ChatAction): ChatState {
  switch (action.type) {
    case 'ADD_MESSAGE':
      return { ...state, messages: [...state.messages, action.payload] };
    case 'UPDATE_MESSAGE':
      return {
        ...state,
        messages: state.messages.map(m =>
          m.id === action.payload.id ? { ...m, ...action.payload.updates } : m
        ),
      };
    case 'SET_LOADING':
      return { ...state, isLoading: action.payload };
    case 'SET_CONVERSATION_ID':
      return { ...state, conversationId: action.payload };
    case 'SET_ERROR':
      return { ...state, error: action.payload };
    case 'CLEAR_MESSAGES':
      return { ...state, messages: [], conversationId: null };
    default:
      return state;
  }
```

```
}

export function ChatProvider({ children }: { children: ReactNode }) {
  const pathname = usePathname();
  const [state, dispatch] = useReducer(chatReducer, {
    messages: [],
    isLoading: false,
    conversationId: null,
    error: null,
  });

  const sendMessage = useCallback(async (content: string) => {
    const userMessage: Message = {
      id: crypto.randomUUID(),
      role: 'user',
      content,
      timestamp: new Date(),
    };
    dispatch({ type: 'ADD_MESSAGE', payload: userMessage });
    dispatch({ type: 'SET_LOADING', payload: true });

    const assistantMessageId = crypto.randomUUID();
    dispatch({
      type: 'ADD_MESSAGE',
      payload: { id: assistantMessageId, role: 'assistant', content: '', timestamp: new
    });

    try {
      const response = await fetch('/api/chatbot/message', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          messages: state.messages.concat(userMessage).map(m => ({
            role: m.role,
            content: m.content,
          })),
          conversationId: state.conversationId,
          currentPage: pathname,
        }),
      });

      const reader = response.body?.getReader();
      const decoder = new TextDecoder();
```

```
if (!reader) throw new Error('No response stream');

let fullContent = '';

while (true) {
  const { done, value } = await reader.read();
  if (done) break;

  const chunk = decoder.decode(value);
  const lines = chunk.split('\n\n');

  for (const line of lines) {
    if (!line.startsWith('data: ')) continue;
    const data = JSON.parse(line.slice(6));

    switch (data.type) {
      case 'text':
        fullContent += data.content;
        dispatch({
          type: 'UPDATE_MESSAGE',
          payload: { id: assistantMessageId, updates: { content: fullContent } },
        });
        break;
      case 'tool_result':
        if (data.tool === 'execute_system_action') {
          dispatch({
            type: 'UPDATE_MESSAGE',
            payload: {
              id: assistantMessageId,
              updates: {
                actionCard: {
                  actionType: data.result.action_type,
                  description: data.result.confirmation_message,
                  params: data.result.params,
                  status: 'pending_confirmation',
                },
              },
            },
          });
        }
        break;
      case 'done':
```

```
            break;
          }
        }
      }
    } catch (error: any) {
      dispatch({ type: 'SET_ERROR', payload: error.message });
    } finally {
      dispatch({ type: 'SET_LOADING', payload: false });
    }
  }, [state.messages, state.conversationId, pathname]);

  const confirmAction = useCallback(async (messageId: string) => {
    const message = state.messages.find(m => m.id === messageId);
    if (!message?.actionCard) return;

    dispatch({
      type: 'UPDATE_MESSAGE',
      payload: {
        id: messageId,
        updates: { actionCard: { ...message.actionCard, status: 'confirmed' } },
      },
    });

    const res = await fetch(`/api/chatbot/actions/${messageId}/confirm`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(message.actionCard),
    });

    const result = await res.json();
    dispatch({
      type: 'UPDATE_MESSAGE',
      payload: {
        id: messageId,
        updates: {
          actionCard: { ...message.actionCard, status: result.success ? 'executed' : 'c
        },
      },
    });
  }, [state.messages]);

  const cancelAction = useCallback((messageId: string) => {
    const message = state.messages.find(m => m.id === messageId);
```

```
    if (!message?.actionCard) return;
    dispatch({
      type: 'UPDATE_MESSAGE',
      payload: {
        id: messageId,
        updates: { actionCard: { ...message.actionCard, status: 'cancelled' } },
      },
    });
  }, [state.messages]);

  const markStepComplete = useCallback((messageId: string, stepOrder: number) => {
    const message = state.messages.find(m => m.id === messageId);
    if (!message?.stepGuide) return;
    const updatedSteps = message.stepGuide.steps.map(s =>
      s.order === stepOrder ? { ...s, completed: true } : s
    );
    dispatch({
      type: 'UPDATE_MESSAGE',
      payload: {
        id: messageId,
        updates: {
          stepGuide: { ...message.stepGuide, steps: updatedSteps, currentStep: stepOrde
        },
      },
    });
  }, [state.messages]);

  const clearChat = useCallback(() => {
    dispatch({ type: 'CLEAR_MESSAGES' });
  }, []);

  return (
    <ChatContext.Provider value={{ state, sendMessage, confirmAction, cancelAction, mar
      {children}
    </ChatContext.Provider>
  );
}

export function useChatbot() {
  const context = useContext(ChatContext);
  if (!context) throw new Error('useChatbot must be used within ChatProvider');
  return context;
}
```

12. Tool Handlers (Backend Execution)

```
// src/lib/chatbot/tool-handlers.ts

import { TenantAuthContext } from '@/lib/server/tenant-context';
import { actionGuides } from './knowledge-base';

export async function handleToolCall(
  toolName: string,
  input: any,
  tenant: TenantAuthContext
): Promise<any> {
  switch (toolName) {
    case 'get_step_by_step_guide':
      return handleGetGuide(input);
    case 'navigate_user':
      return handleNavigate(input);
    case 'execute_system_action':
      return handleExecuteAction(input, tenant);
    case 'query_system_data':
      return handleQueryData(input, tenant);
    case 'search_help':
      return handleSearchHelp(input);
    default:
      return { error: `Unknown tool: ${toolName}` };
  }
}

function handleGetGuide(input: { action: string; user_role?: string }) {
  const query = input.action.toLowerCase();
  const matched = actionGuides.filter(g =>
    g.title.toLowerCase().includes(query) ||
    g.id.includes(query.replace(/\s+/g, '_'))
  );

  if (matched.length === 0) {
    return {
      found: false,
      message: 'No exact guide found. Let me provide general guidance based on the requ
      suggestions: actionGuides.slice(0, 5).map(g => g.title),
    };
  }
```

```
  const guide = matched[0];
  return {
    found: true,
    guide: {
      title: guide.title,
      titleAr: guide.titleAr,
      module: guide.module,
      requiredRoles: guide.requiredRoles,
      route: guide.route,
      steps: guide.steps,
      stepsAr: guide.stepsAr,
    },
  };
}

function handleNavigate(input: { destination: string; reason?: string }) {
  const routeMap: Record<string, string> = {
    employees: '/employees',
    departments: '/departments',
    locations: '/locations',
    attendance: '/attendance',
    leaves: '/leaves',
    tasks: '/tasks',
    payroll: '/payroll',
    recruitment: '/recruitment',
    sales: '/sales',
    assets: '/assets',
    accounting: '/accounting',
    settings: '/settings',
    chat: '/chat',
    notifications: '/notifications',
    schedule: '/schedule',
  };

  const key = input.destination.toLowerCase();
  const route = routeMap[key] || `/${key}`;

  return {
    action: 'navigate',
    route,
    reason: input.reason || `Navigate to ${input.destination}`,
  };
```

```
}

async function handleExecuteAction(
  input: { action_type: string; params: any; confirmation_message: string },
  tenant: TenantAuthContext
) {
  // Map action types to internal API calls
  const actionMap: Record<string, (params: any, tenant: TenantAuthContext) => Promise<a
    create_employee: async (params, t) => {
      const { data, error } = await t.admin
        .from('employees')
        .insert({ ...params, organization_id: t.orgId })
        .select()
        .single();
      return error ? { success: false, error: error.message } : { success: true, data }
    },
    submit_leave_request: async (params, t) => {
      const { data, error } = await t.admin
        .from('leave_requests')
        .insert({
          ...params,
          organization_id: t.orgId,
          employee_id: params.employee_id || t.userId,
          status: 'pending',
        })
        .select()
        .single();
      return error ? { success: false, error: error.message } : { success: true, data }
    },
    approve_leave_request: async (params, t) => {
      const { error } = await t.admin.rpc('execute_leave_review_command', {
        p_request_id: params.request_id,
        p_reviewer_id: t.userId,
        p_action: 'approve',
        p_rejection_note: null,
      });
      return error ? { success: false, error: error.message } : { success: true };
    },
    create_task: async (params, t) => {
      const { data, error } = await t.admin
        .from('tasks')
        .insert({
          ...params,
```

```
          organization_id: t.orgId,
          created_by: t.userId,
        })
        .select()
        .single();
      return error ? { success: false, error: error.message } : { success: true, data }
    },
    // ... Additional action handlers
  };

  const handler = actionMap[input.action_type];
  if (!handler) {
    return { success: false, error: `Action type "${input.action_type}" not yet impleme
  }

  return {
    action_type: input.action_type,
    params: input.params,
    confirmation_message: input.confirmation_message,
    requires_confirmation: true,
  };
}

async function handleQueryData(
  input: { query_type: string; filters?: any },
  tenant: TenantAuthContext
) {
  const { admin, orgId, userId } = tenant;

  switch (input.query_type) {
    case 'employee_count': {
      const { count } = await admin
        .from('employees')
        .select('*', { count: 'exact', head: true })
        .eq('organization_id', orgId);
      return { count };
    }
    case 'leave_balance': {
      const { data } = await admin
        .from('leave_balances')
        .select('*')
        .eq('employee_id', input.filters?.employee_id || userId);
      return { balances: data };
```

```
    }
    case 'attendance_today': {
      const today = new Date().toISOString().split('T')[0];
      const { data } = await admin
        .from('attendance')
        .select('*')
        .eq('organization_id', orgId)
        .gte('check_in', `${today}T00:00:00`)
        .lte('check_in', `${today}T23:59:59`);
      return { records: data, count: data?.length || 0 };
    }
    case 'pending_approvals': {
      const { data } = await admin
        .from('leave_requests')
        .select('*, employees(full_name)')
        .eq('organization_id', orgId)
        .eq('status', 'pending');
      return { pending: data, count: data?.length || 0 };
    }
    case 'pending_tasks': {
      const { data } = await admin
        .from('tasks')
        .select('*')
        .eq('organization_id', orgId)
        .in('status', ['open', 'in_progress']);
      return { tasks: data, count: data?.length || 0 };
    }
    default:
      return { error: `Query type "${input.query_type}" not yet implemented` };
  }
}

function handleSearchHelp(input: { query: string }) {
  const query = input.query.toLowerCase();
  const results = actionGuides
    .filter(g =>
      g.title.toLowerCase().includes(query) ||
      g.module.toLowerCase().includes(query) ||
      g.steps.some(s => s.instruction.toLowerCase().includes(query))
    )
    .slice(0, 5);

  return {
```

```
    results: results.map(g => ({
      title: g.title,
      titleAr: g.titleAr,
      module: g.module,
      route: g.route,
    })),
  };
}
```

13. Environment Variables

Add to `.env.local`:

```
# Anthropic Claude API
ANTHROPIC_API_KEY=sk-ant-xxxxxxxxxxxx

# Chatbot Configuration
CHATBOT_MODEL=claude-sonnet-5
CHATBOT_MAX_TOKENS=4096
CHATBOT_RATE_LIMIT_PER_MINUTE=20
CHATBOT_HISTORY_LIMIT=50
```

14. Implementation Phases

Phase 1: Foundation (Week 1-2)

- [ ] Install `@anthropic-ai/sdk` package
- [ ] Create database migration (`schema_v137_chatbot.sql`)
- [ ] Run migration
- [ ] Create `/api/chatbot/message` route with streaming
- [ ] Build `ChatWidget` and `ChatPanel` components
- [ ] Add `ChatProvider` context
- [ ] Integrate widget into the main app layout
- [ ] Basic text conversation working

Phase 2: Knowledge & Guidance (Week 2-3)

- [ ] Build complete `knowledge-base.ts` with all action guides (Arabic + English)
- [ ] Implement `get_step_by_step_guide` tool handler
- [ ] Implement `search_help` tool handler
- [ ] Implement `navigate_user` tool handler
- [ ] Create `StepGuide` component with progress tracking
- [ ] Add suggested questions based on current page

Phase 3: Data Queries (Week 3-4)

- [ ] Implement `query_system_data` tool handler (all query types)
- [ ] Add data visualization in chat (mini tables, counts, lists)
- [ ] Implement conversation history persistence
- [ ] Add conversation list/archive functionality

Phase 4: Action Execution (Week 4-5)

- [ ] Implement `execute_system_action` tool handler
- [ ] Build `ActionCard` component with confirm/cancel
- [ ] Add action audit logging
- [ ] Implement action confirmation API
- [ ] Add permission validation before action execution
- [ ] Test all action types end-to-end

Phase 5: Polish & Production (Week 5-6)

- [ ] Add rate limiting specific to chatbot
- [ ] Implement conversation archiving/cleanup
- [ ] Add typing indicators and loading states
- [ ] Mobile-responsive chat widget
- [ ] RTL support for Arabic
- [ ] Error handling and fallback responses
- [ ] Performance optimization (debouncing, message batching)
- [ ] Add analytics tracking for chatbot usage

15. Security Considerations

Concern	Mitigation
Prompt injection	System prompt is server-side only; user input is in message role
Data access	All queries use tenant context; RLS enforced on all tables
Action execution	Two-step confirm pattern; audit log for all actions
API key exposure	Key stored in env variable, never sent to client
Rate limiting	Per-user rate limit (20 messages/min default)
Token cost	Max token limit per response; conversation history capped
PII in logs	Message content encrypted at rest; audit log stores action metadata only

16. Cost Estimation

Component	Monthly Cost (Est.)
Claude API (Sonnet, ~50 users, ~20 msgs/day each)	~$150-300
Supabase storage (conversations)	Included in plan
Additional compute	Minimal (streaming)

17. Quick Reference: Integration Points

```
// Add to src/app/(app)/layout.tsx
import { ChatWidget } from '@/components/chatbot/ChatWidget';

export default function AppLayout({ children }) {
  return (
    <>
      {children}
      <ChatWidget />
    </>
  );
}


# Install dependency
npm install @anthropic-ai/sdk
```

18. Example Conversations

Example 1: Step-by-Step Guidance

```
User: How do I add a new employee?
Bot: Here's how to add a new employee:

📋 **Add a New Employee** (Module: HR)
Required Role: Admin, HR_Admin

1. Navigate to "Employees" from the sidebar menu
2. Click the "Add Employee" button in the top right
3. Fill in the required fields:
   - Full Name
   - Employee ID
   - Department
   - Position
   - Hire Date
4. Optionally fill in: Phone, Email, Nationality, ID Number
5. Set salary details: Basic Salary, Housing Allowance, Transport Allowance
6. Click "Save" to create the employee record

💡 Tip: If you provide an email, the system will automatically create a login account a

Would you like me to take you to the Employees page, or would you like me to walk you t
```

Example 2: Action Execution

User: Create a task for Ahmed to submit the monthly report by next Sunday
Bot: I'll create that task for you. Here's what I'll do:

```
┌──────────────────────────────────────┐
│ 🔧 Create Task                       │
│                                      │
│ Title: Submit Monthly Report         │
│ Assigned to: Ahmed                   │
│ Due date: 2026-08-06                 │
│ Priority: Medium                     │
│                                      │
│ [✓ Confirm]  [✗ Cancel]              │
└──────────────────────────────────────┘
```

User: *clicks Confirm*
Bot: ✓ Task created successfully! Ahmed will receive a notification about this task.

Example 3: Arabic Support

User: كيف أقدم طلب إجازة؟
Bot: إليك خطوات تقديم طلب إجازة:

📋 **تقديم طلب إجازة**

١. انتقل إلى "الإجازات" من القائمة الجانبية
٢. اضغط على زر "طلب إجازة"
٣. اختر نوع الإجازة (سنوية، مرضية، طارئة، إلخ)
٤. حدد تاريخ البداية وتاريخ النهاية
٥. أضف سبب/ملاحظة للإجازة
٦. اضغط "إرسال" لتقديم الطلب للموافقة

💡 نصيحة: ستتلقى إشعارًا عند اتخاذ القرار من مديرك أو الموارد البشرية

هل تريد أن آخذك إلى صفحة الإجازات، أو تريدني أقدم الطلب عنك؟

19. Future Enhancements (Post-MVP)

- **Voice input**: Add speech-to-text for hands-free operation
- **Proactive suggestions**: Widget shows contextual tips based on current page
- **Analytics dashboard**: Track most-asked questions to improve UX
- **Custom training**: Fine-tune responses based on organization-specific policies
- **Multi-modal**: Accept screenshots for troubleshooting
- **Workflow automation**: Chain multiple actions into workflows
- **Integration with notifications**: Bot alerts users about pending actions

Document Version: 1.0
Created: 2026-07-30
System: Tatwir Talent HRMS